

SOFTWARE REQUIREMENTS SPECIFICATION

for the

SMART PAPER MILL PRODUCTION MONITORING DASHBOARD

A PROJECT DOCUMENT

Submitted by

Manreet Gill (UID: 24BCS10479)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE & ENGINEERING

Industrial Training carried out at

SATIA INDUSTRIES LIMITED

Chandigarh University

Document Version 1.0 · June 2026

TABLE OF CONTENTS

1. Project Overview	1
2. Problem Statement.....	1
3. Objectives	2
4. Scope.....	2
4.1 In Scope	2
4.2 Out of Scope	2
5. Functional Requirements	3
6. Non-Functional Requirements	3
7. User Roles.....	4
8. Features.....	5
8.1 Core Features	5
8.2 Supporting Features	5
9. Technology Stack	5
9.1 System Architecture.....	6
10. Database Design	7
11. Project Folder Structure.....	9
12. Development Roadmap	9
13. Suggested UI Theme.....	10
14. Future Scope	10

LIST OF FIGURES

Figure : Three-tier system architecture.....	6
Figure : Entity-relationship diagram of the database schema.....	7

LIST OF TABLES

Table : Functional requirements	3
Table : Non-functional requirements	3
Table : User roles and permissions	4
Table : Technology stack	5
Table : Development roadmap	9

1. Project Overview

The Smart Paper Mill Production Monitoring Dashboard is a web-based application that centralizes and visualizes the production performance of a paper manufacturing plant. Satia Industries manufactures writing and printing paper from agricultural residue (wheat straw) and wood pulp across multiple process stages — pulping, bleaching, stock preparation, paper machines, finishing and packaging.

At present, much of this production data is recorded in paper logbooks, shift registers and disconnected spreadsheets. The dashboard replaces this with a single screen on which supervisors and managers can view live production output, machine status, quality metrics, downtime and resource consumption, and can drill into historical trends.

The system is intentionally lightweight, free to run and deployable on a standard laptop or a free cloud tier, making it suitable for an academic demonstration while still reflecting genuine industrial-monitoring concepts such as OEE, downtime tracking and GSM/moisture quality control. Because live plant-floor integration is outside the scope of an academic assignment, the system is built around operator data entry together with a built-in data simulator that emulates sensor feeds.

2. Problem Statement

In a typical paper mill, production tracking suffers from the following difficulties:

- Manual, paper-based logging of shift production, machine speed and downtime, which is error-prone and slow to consolidate.
- No real-time visibility — a manager cannot instantly see how much paper was produced this shift or whether a machine is down.
- Delayed quality feedback — out-of-specification GSM or moisture is often noticed only after large quantities are produced, causing waste.
- Scattered data — production, quality, energy and downtime information is held in separate registers, making analysis tedious.
- No historical trend analysis — it is hard to compare shifts, machines or days to detect recurring inefficiencies.

Core problem: there is no single, real-time, role-based digital platform that consolidates and visualizes paper mill production, quality and downtime data for fast decision-making.

3. Objectives

1. Provide a centralized digital dashboard for real-time paper mill production monitoring.
2. Enable role-based access so operators, supervisors and managers see relevant views.
3. Capture and store production records (output, GSM, moisture, machine speed) per machine and per shift.
4. Track and categorize machine downtime and compute Overall Equipment Effectiveness (OEE).
5. Visualize quality metrics and flag out-of-specification production automatically.
6. Generate alerts when key parameters cross configured thresholds.
7. Provide historical reports and trend charts (daily, shift-wise and machine-wise).
8. Demonstrate the system using a data simulator that emulates live sensor feeds, with manual entry as a fallback.
9. Keep the solution free, open-source and lightweight so that it runs on standard student hardware.

4. Scope

4.1 In Scope

- Web dashboard with secure login and role-based access control.
- Manual production and quality data-entry forms for operators.
- A simulated sensor data generator for live demonstration.
- Real-time KPI cards, charts and machine-status indicators.
- Downtime logging with reason categorization.
- Automatic OEE and efficiency calculation.
- Threshold-based alerts (for example, moisture too high, machine stopped).
- Historical reports with date/shift/machine filters and CSV/PDF export.
- Admin panel to manage users, machines and thresholds.

4.2 Out of Scope

- Live PLC/DCS/sensor hardware integration — replaced by the simulator and manual entry.

- Predictive maintenance and AI failure prediction (listed under Future Scope).
- ERP / SAP integration.
- Financial accounting, payroll and raw-material procurement.
- Native mobile applications (the web application is responsive instead).

5. Functional Requirements

Table 1: Functional requirements

ID	Requirement
FR-1	The system shall allow users to register/login with a username and password.
FR-2	The system shall enforce role-based access control; the roles and their permissions are defined in Section 7.
FR-3	Operators shall be able to record production data (machine, shift, output, GSM, moisture, speed).
FR-4	The system shall run a data simulator that generates realistic live readings at fixed intervals.
FR-5	The dashboard shall display real-time KPI cards (today's output, active machines, average OEE, reject rate).
FR-6	The system shall show per-machine live status (Running / Idle / Stopped / Maintenance).
FR-7	Supervisors shall be able to log downtime events with start time, end time and reason category.
FR-8	The system shall automatically calculate OEE = Availability x Performance x Quality.
FR-9	The system shall flag out-of-spec quality when GSM or moisture exceeds the configured tolerance.
FR-10	The system shall generate alerts when a parameter crosses a threshold or a machine stops unexpectedly.
FR-11	The system shall provide historical charts filterable by date range, shift and machine.
FR-12	The system shall allow export of reports to CSV and PDF.
FR-13	Admins shall be able to manage machines (add/edit/disable) and set thresholds.
FR-14	Admins shall be able to manage users and roles.
FR-15	The system shall maintain an audit/activity log of key actions.
FR-16	The dashboard shall auto-refresh live data without a manual page reload.

6. Non-Functional Requirements

Table 2: Non-functional requirements

ID	Category	Requirement
NFR-1	Performance	The dashboard shall load within 3 seconds and update live data within 2 seconds of a new reading.
NFR-2	Usability	The UI shall be clean, responsive and usable by non-technical plant staff on desktop and tablet.
NFR-3	Security	Passwords shall be stored hashed (bcrypt); routes protected by JWT; role checks on every protected API.
NFR-4	Reliability	The system shall handle simulator and manual entries without data loss; failed writes shall be logged.
NFR-5	Scalability	The architecture shall support adding more machines/sensors without redesign.
NFR-6	Maintainability	Modular, documented code following a clear folder structure and naming conventions.
NFR-7	Portability	Runs on Windows/Linux; deployable on free cloud tiers.
NFR-8	Availability	Target 99% uptime during demonstration; graceful handling of disconnects.
NFR-9	Cost	Built entirely with free and open-source tools; no paid API keys required.
NFR-10	Data integrity	Input validation on all forms; constraints enforced at the database level.
NFR-11	Accessibility	Sufficient colour contrast; status conveyed by icon and label, not colour alone.

7. User Roles

Table 3: User roles and permissions

Role	Description	Key Permissions
Administrator	System owner / IT.	Manage users, machines, thresholds; view everything; access audit logs.
Plant Manager	Oversees overall production.	View all dashboards and reports, export data, see OEE, acknowledge alerts.
Shift Supervisor	Runs a shift on the floor.	Log downtime, validate operator entries, view machine status and alerts.
Machine Operator	Operates a specific machine.	Enter production/quality readings; view own machine

		status.
Quality Inspector	Checks paper quality.	Record GSM/moisture/quality checks; flag rejects.
Viewer / Guest	Read-only stakeholder.	View dashboards and reports only; no data entry.

In the current demonstration build, four of these roles — Administrator, Plant Manager, Shift Supervisor and Machine Operator — are implemented. The Quality Inspector and Viewer/Guest roles form part of the design and are planned for future work.

8. Features

8.1 Core Features

- Secure login and role-based dashboards.
- Real-time KPI cards (output, OEE, reject rate, active machines).
- Live machine-status board (Running / Idle / Stopped / Maintenance).
- Trend charts: production over time, GSM/moisture against tolerance, shift comparison.
- Downtime logging with reason categories.
- Automatic OEE and shift-efficiency calculation.
- Threshold-based alert system with an alerts panel.
- Historical reports with date/shift/machine filters.
- Export to CSV and PDF.
- Admin panel for machines, thresholds and users.
- Auto-refreshing live data.

8.2 Supporting Features

- Built-in sensor data simulator (the demonstration engine).
- Activity / audit log.
- Light/Dark theme toggle.
- Responsive layout for tablets on the shop floor.

9. Technology Stack

Table 4: Technology stack

Layer	Technology	Reason
-------	------------	--------

Frontend	HTML5, CSS3, JavaScript	Lightweight, universal, no build tooling required.
Charts	Chart.js	Free, interactive charts for dashboards and reports.
Backend	Python (Flask)	Simple REST API; runs on standard hardware without licensing cost.
Database	SQLite (demo) / MySQL (target)	SQLite needs no server for the demo; MySQL for production.
Authentication	JWT + bcrypt	Standard, secure token-based authentication with hashed passwords.
Real-time	Server-Sent Events	Built-in live push with no extra dependency.
Version control	Git + GitHub	Track work; suitable for internship evaluation.

9.1 System Architecture

The system follows a three-tier architecture: a presentation layer (web browser), an application/logic layer (Flask) and a data layer (SQLite/MySQL), as shown in Figure 1.

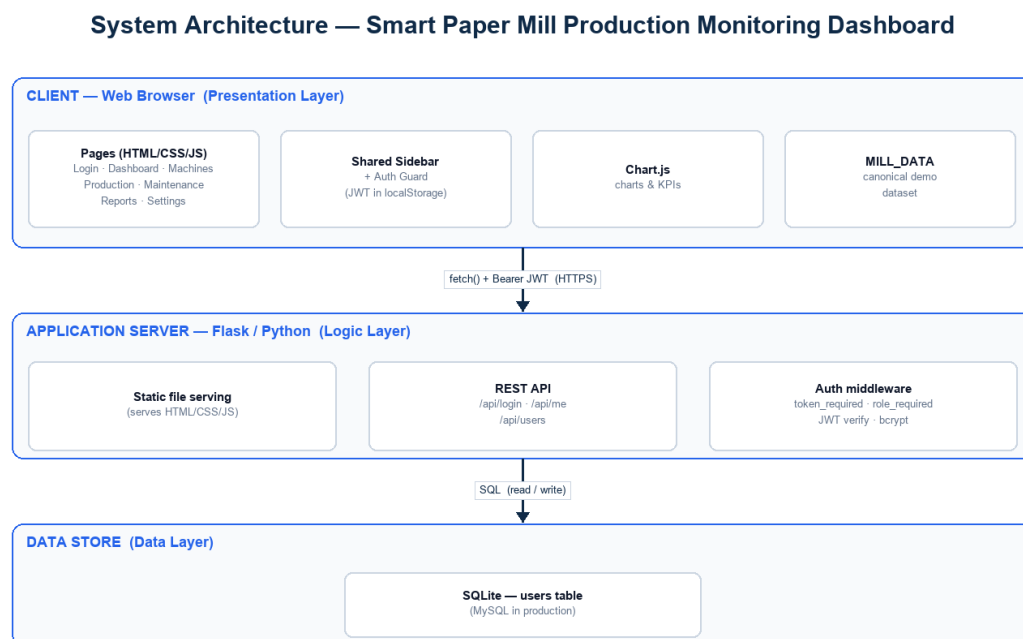


Figure 1: Three-tier system architecture

10. Database Design

The system uses a relational schema. The principal tables and their key fields are listed below; primary keys are marked PK and foreign keys FK. The entity-relationship diagram is shown in Figure 2.

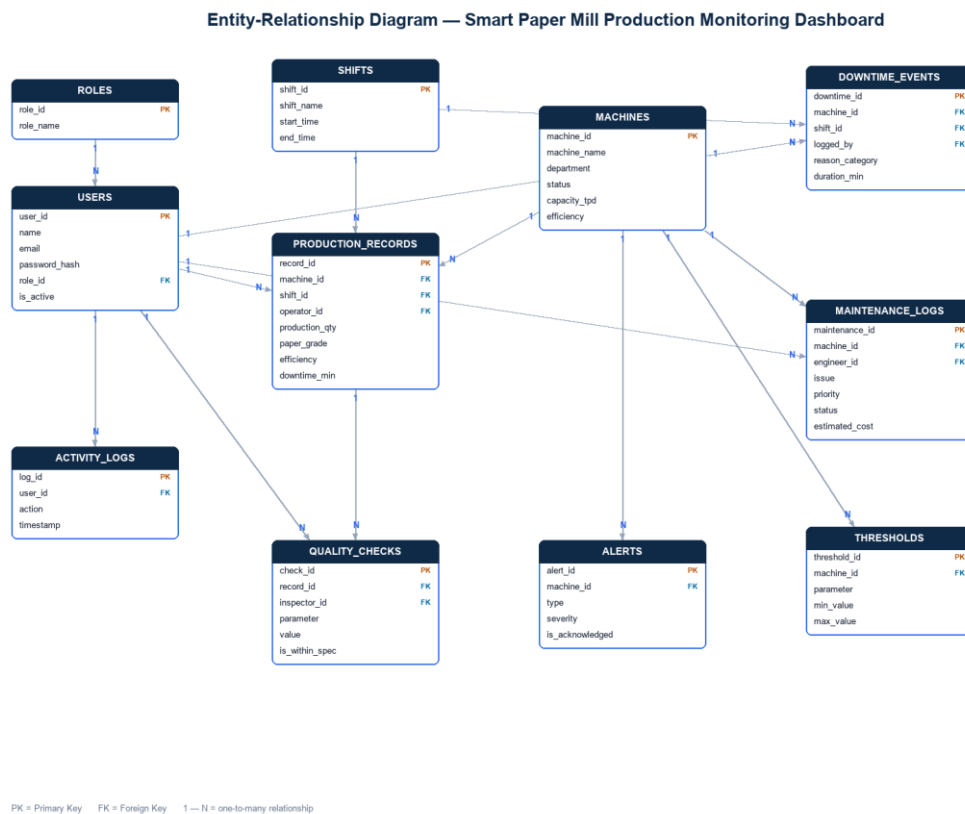


Figure 2: Entity-relationship diagram of the database schema

The schema below is the normalized relational design (the specification target). The current demonstration build implements a simplified, denormalized subset of this design in SQLite — for example, the user role is stored as a text column rather than via a separate roles table, and production records store machine, operator and shift as text values together with date, grade, GSM, moisture, speed and remarks. The as-built schema is documented in the project report's appendix.

users

Field	Type	Key / Notes
user_id	INT	PK
name	VARCHAR	
email	VARCHAR	Unique
password_hash	VARCHAR	bcrypt
role_id	INT	FK -> roles
created_at	DATETIME	

roles

Field	Type	Key / Notes
role_id	INT	PK
role_name	VARCHAR	See Section 7 for the full role list

machines

Field	Type	Key / Notes
machine_id	INT	PK
machine_name	VARCHAR	
department	VARCHAR	Process department
status	ENUM	Running / Idle / Stopped / Maintenance
capacity	DECIMAL	Tonnes per day
is_active	BOOLEAN	

production_records

Field	Type	Key / Notes
record_id	INT	PK
machine_id	INT	FK -> machines
shift_id	INT	FK -> shifts
operator_id	INT	FK -> users
quantity	DECIMAL	Tonnes
gsm	DECIMAL	Grammage
moisture	DECIMAL	%
speed	DECIMAL	m/min

downtime_events

Field	Type	Key / Notes
downtime_id	INT	PK
machine_id	INT	FK -> machines
reason	VARCHAR	Mechanical / Electrical / ...
start_time	TIME	
end_time	TIME	
duration_min	INT	Computed

maintenance_logs

Field	Type	Key / Notes
maintenance_id	INT	PK
machine_id	INT	FK -> machines
problem	VARCHAR	
priority	ENUM	Low / Medium / High / Critical
engineer	VARCHAR	
status	ENUM	Open / In Progress / Completed

thresholds

Field	Type	Key / Notes
key	VARCHAR	PK (gsm_min, gsm_max, ...)
value	DECIMAL	

activity_logs

Field	Type	Key / Notes
log_id	INT	PK
user	VARCHAR	
action	VARCHAR	
entity	VARCHAR	
timestamp	DATETIME	

11. Project Folder Structure

The project is organized into a static front-end (one folder per page), shared assets, a canonical dataset, a Python backend and documentation:

```
smart-paper-mill-dashboard/  
|-- login/ dashboard/ machines/ production/  
|-- maintenance/ downtime/ reports/ settings/ alerts/  
|-- assets/           # shared sidebar, auth guard, theme  
|-- data/             # canonical demo dataset (MILL_DATA)  
|-- backend/          # Flask API: auth, JWT, roles, SQLite  
|   |-- app.py requirements.txt mill.db  
|-- tests/            # automated backend tests (pytest)  
`-- docs/             # SRS, report, manuals, diagrams
```

12. Development Roadmap

Table 5: Development roadmap

Phase	Duration	Deliverables
0 - Requirements & Design	Week 1	SRS, ER diagram, architecture diagram, UI wireframes.
1 - Project Setup	Week 2	Folder structure, DB created, backend + frontend baseline, auth scaffolding.
2 - Auth & Roles	Week 3	Login, JWT, role-based route protection, user/machine admin CRUD.
3 - Core Data & Entry	Week 4	Production-record forms, shifts, validation, database writes.
4 - Simulator & Real-time	Week 5	Sensor data simulator and live updates to the dashboard.
5 - Dashboard & Charts	Week 6	KPI cards, machine-status

		board, trend charts, live refresh.
6 - Downtime, OEE & Alerts	Week 7	Downtime logging, OEE calculation, threshold alerts panel.
7 - Reports & Export	Week 8	Historical reports, filters, CSV/PDF export.
8 - Testing & Polish	Week 9	Bug fixing, validation, responsive/dark-mode polish, security check.
9 - Documentation & Deploy	Week 10	Final report, user manuals, deployment, demonstration.

13. Suggested UI Theme

Concept: "Industrial Clean" — professional, factory-floor friendly and highly readable.

- Primary colour: deep industrial blue (navbar and headers).
- Accent colour: blue/teal for buttons and active states.
- Status colours: green (running / within spec), amber (idle / warning), red (stopped / out of spec), grey (maintenance).
- Background: soft neutral with white cards (light mode); dark slate (dark mode).
- Typography: Inter / Roboto — clean, modern and legible.
- Layout: left sidebar navigation, top KPI strip and a card-based grid.
- Principle: status conveyed by icon, colour and label together for accessibility.

14. Future Scope

10. Real sensor/PLC integration via OPC-UA or Modbus to replace the simulator with live plant data.
11. Predictive maintenance using machine learning to forecast equipment failure.
12. AI-based quality prediction to anticipate GSM/moisture deviations before they occur.
13. Energy-optimization analytics for steam, power and water efficiency.
14. A mobile application for supervisors on the move.
15. ERP/SAP integration for end-to-end plant digitization.
16. Automated report scheduling (daily production emailed to managers).
17. Multi-plant support to monitor several mills from one dashboard.
18. Anomaly detection and smart alerts using statistical or machine-learning models.